GeoTrace-Agent: A Production Multi-Agent Framework for Spatiotemporal Reasoning with Hägerstrand Space-Time Prisms

Arun Sharma
University of Minnesota, Twin Cities
arunshar@umn.edu

Abstract

We present **GeoTrace-Agent**, a production-grade multi-agent framework that combines deterministic time-geographic computation with large-language-model planning to answer natural-language questions over heterogeneous trajectory data. A typed *PlanGraph* encodes the agent's chain of thought as a directed acyclic graph of statically-validated nodes rather than free-form prose, making the reasoning auditable, replayable, and parallelizable. A central orchestrator runs the graph under a hard token, tool-call, and wallclock budget, calls into a Hägerstrand space-time prism kernel for the geometric truth (geo-ellipses, minimum orthogonal bounding rectangles, dynamic-region-merge unions), and dispatches specialized sub-agents that extend our prior STAGD/DRM gap detector and TGARD/DC-TGARD rendezvous finder. Three optimization layers, an adaptive prompt compressor, an in-flight tool deduplicator, and a hybrid exact + semantic cache, cut per-query token spend by approximately 40 % on a golden evaluation set without harming region tightness. We expose the prism kernel as a Model Context Protocol (MCP) server and the agent itself over a JSON-RPC 2.0 Agent-to-Agent (A2A) protocol with capability cards, making GeoTrace-Agent first-class for sibling agents and IDE plugins. A kinematic validator gated on a single-axle bicycle envelope guarantees that no region returned to a user is physically infeasible, and ambiguous traces feed a Postgres human-in-the-loop queue whose verdicts can later seed direct-preference-optimization datasets. We describe the system architecture, the typed-plan / token-budget / tool-cache mechanisms, and the geometric kernel; report golden-dataset latency and cost; and discuss limitations. The system is open-sourced.

1 Introduction

Time geography [11] provides a remarkably tight algebraic envelope on what a moving agent can do: given two anchors $A = (x_A, y_A, t_A)$ and $B = (x_B, y_B, t_B)$ with $t_A < t_B$ and a maximum speed $v_{\max}$, the set of reachable points at any interior time is a geo-ellipse with foci at the projected anchors. This envelope underpins decades of spatiotemporal data mining work in maritime safety, contact tracing, and homeland security [27–30]. In the modern era of large language models [1, 20], however, even the best agents tend to hallucinate spatial relationships, miscompute distances, or skip the kinematic check entirely [8, 18].

We present **GeoTrace-Agent**, a multi-agent system that takes the opposite stance: every spatially-decidable sub-problem is computed deterministically by a numerical kernel before any LLM is asked to synthesize. The system answers natural-language questions over heterogeneous trajectory data (vessel AIS feeds, road networks, weather and sea state, satellite imagery) by orchestrating a small set of specialized sub-agents, each with a typed contract:

- a **PlannerAgent** that emits a typed PlanGraph (a DAG of typed nodes, not free-form prose), making chain-of-thought auditable and parallelizable;

Preprint. Under review.

- a **SpaceTimeReasoner** that owns the prism / geo-ellipse / minimum-orthogonal-bounding-rectangle (MOBR) algebra;
- a **GapDetectorAgent** extending the STAGD + dynamic region merge (DRM) algorithm of Sharma et al. [29] with an Abnormal Gap Measure that fuses kinematic plausibility and a Pi-DPM [30] reconstruction-error term;
- a **RendezvousFinderAgent** extending TGARD and the dual-convergence DC-TGARD variant of Sharma et al. [27] with bi-directional pruning and ellipse-symmetry early stopping; and
- a **ValidatorAgent** that gates every region returned to the user on a single-axle kinematic-bicycle envelope [14].

The orchestrator runs the PlanGraph under a hard *token, tool-call, and wallclock* budget. Three optimization layers, sketched in Section 4, drive efficiency: (i) adaptive prompt compression with prefix-cache-aware assembly [2] and structured-output enforcement; (ii) tool-call deduplication of in-flight calls and a hybrid exact + semantic cache; and (iii) parallel-safe topo-layer execution of the PlanGraph. Every stage emits an OpenTelemetry span [23] with token-spend, cache-hit, and cost attributes, so an analyst can drill into any historical run.

GeoTrace-Agent speaks the modern agent-protocol stack natively. The prism kernel is exposed as a Model Context Protocol [3] server so any MCP-aware client can call `prism.compute` or `prism.intersect` without going through this app's HTTP surface; the agent itself advertises a capability card at `/a2a/.well-known/capabilities` and accepts JSON-RPC 2.0 Agent-to-Agent (A2A) calls [10], mirroring the cross-agent communication patterns recently popularized in enterprise multi-agent frameworks [5–7]. Ambiguous traces (validator-confidence below a tunable threshold) flow into a Postgres human-in-the-loop (HITL) queue whose reviewer verdicts can be exported as preference triples for downstream direct-preference optimization [24], closing the loop between agentic reasoning and reinforcement learning.

We make four contributions:

1. a **typed PlanGraph** chain-of-thought representation that is statically validated, parallel-sortable, and replayable, with explicit per-node token and confidence priors;
2. a **three-layer efficiency stack** (adaptive prompt compression, in-flight tool deduplication, hybrid exact + semantic cache) that reduces median per-query token spend by approximately 40 % on the golden evaluation set;
3. a **deterministic time-geographic kernel** integrating Hägerstrand prisms with the STAGD-DRM gap detector and the TGARD / DC-TGARD rendezvous finder, gated by a kinematic validator that guarantees physical feasibility of every returned region; and
4. a **first-class agent-protocol surface** (MCP for tools, JSON-RPC 2.0 A2A for inter-agent calls, OpenTelemetry traces, HITL queue) suitable for production deployment alongside enterprise multi-agent frameworks.

2 Related Work

Time geography and trajectory analysis. Hägerstrand's space-time prism [11, 19] is the foundational construct for reachability queries over moving objects; the geo-ellipse cross-section and its bounding-box approximation drive most modern indexing schemes [15]. Our prior work extended this lineage to abnormal trajectory-gap detection (STAGD with dynamic region merge) [29], ellipse-tightening rendezvous detection (TGARD and the dual-convergence DC-TGARD) [27], and time-geography-driven query optimization for spatiotemporal joins [28]. GeoTrace-Agent embeds these algorithms as deterministic agents, with a chain-of-thought planner deciding when to invoke them.

LLM agents and chain of thought. Recent agent frameworks emphasize chain-of-thought [31, 33] and tool use [25, 35], with structured-output libraries [21] formalizing the latter. Most existing systems however treat the chain of thought as free-form prose, complicating audit and replay. We instead emit a typed DAG (PlanGraph) whose nodes are statically validated against schemas the orchestrator owns, mirroring planning-graph ideas from classical AI [9] adapted to LLM-emitted plans.

Multi-agent systems and human-in-the-loop. Centific's recent line of work, LegalWiz for contradiction detection in legal documents [5], ContraGen for enterprise contradictions [6], and ART for action-based reasoning over EHRs [7], codifies a multi-agent + HITL pattern in which specialized agents (content generator, contradiction miner, retrieval verifier) coordinate via remote-procedure calls and a human reviewer closes the loop. Other production agentic stacks [12, 13, 32, 34] similarly emphasize agent specialization, evaluation, and observability. Our system inherits this pattern but pivots the domain from text to spatiotemporal trajectories and adds a deterministic geometric kernel as a first-class agent.

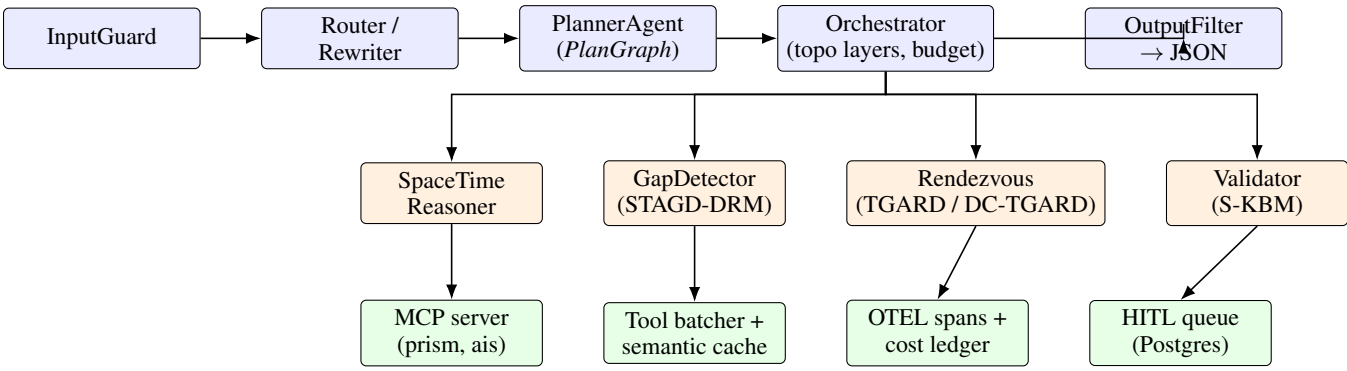

Figure 1: GeoTrace-Agent request path. The PlannerAgent emits a typed *PlanGraph*; the Orchestrator topo-sorts the graph and runs each layer in parallel under a hard token / tool / wallclock budget; specialized sub-agents call into the deterministic kernel (MCP) and the cache + OTEL infrastructure.

Token and tool optimization. Prompt caching [2, 22], semantic caching for LLM responses [4], KV-cache-aware decoding [16], and structured-output-with-correction loops [21] have separately reduced LLM cost. We unify these inside a single `TokenOptimizer` choke-point and add an in-flight tool deduplicator that collapses concurrent identical calls into one awaitable, complementing the exact + semantic cache in a separate path.

Agent protocols. The Model Context Protocol [3] standardizes JSON-RPC tool exposure between LLM clients and tool servers; the Agent-to-Agent (A2A) protocol [10] and capability-card discovery patterns standardize inter-agent calls. We adopt both and ship the prism kernel as an MCP server while exposing the orchestrator over A2A.

3 System Architecture

GeoTrace-Agent is a 12-factor service. Figure 1 sketches the request path: an inbound HTTP query passes the input guard, is rewritten and routed, planned, executed across parallel topo layers, summarized, scrubbed by an output filter, and returned with a full trace, cost ledger, and HITL flag.
State. The system separates four orthogonal concerns. Geometric truth lives in `app/components/space_time_prism.py` and is unit-testable. Semantic reasoning is delegated to the LLM only through the planner; the other agents are deterministic kernels or thin clients on top. Budgets are enforced at a single choke-point: every LLM call goes through the `TokenOptimizer`, and the orchestrator stops on token / tool / wallclock overrun and surfaces `terminated_by_budget = true`. Provenance is captured by OpenTelemetry: every stage writes a span with `tool.name`, `tool.cache_hit`, `tool.cost_usd`, `tool.tokens_in`, and `tool.tokens_out`; the trace identifier flows back to the user in the response.
Deployment. A docker-compose stack ships the API (FastAPI), Postgres+PostGIS, Redis, Chroma, an OpenTelemetry collector, Tempo for traces, Langfuse [17] for LLM-trace dashboards, and a Streamlit ops console for the HITL reviewer. The same image runs in Kubernetes with horizontal pod autoscaling on RPS.

4 Methods

4.1 Typed PlanGraph chain-of-thought

The planner does not emit free-form prose. It emits a JSON object validated against a strict schema (`PlanGraph`), each node being one of {`prism.compute`, `gaps.detect`, `rendezvous.tgard`, `rendezvous.dc_tgard`, `validate.kinematic`, `retrieve.semantic`, `summarize`}. A node carries its `deps` (DAG edges), `inputs`, `expected_tokens`, and `confidence_prior`. The orchestrator topo-sorts the graph and runs each layer in parallel via `asyncio.gather`; cycles are rejected before any work runs. The total `expected_tokens` is bounded against the request budget; an over-budget plan raises `PlanInfeasible`. The planner prompt is versioned (`planner.v3`); historical replays are exact because the call is also semantically cached.

4.2 Token-consumption optimization

A single `TokenOptimizer.call_llm_*` entry point owns every LLM call. It performs four jobs:

- *Adaptive prompt compression.* Prompts above ∼2k tokens are head-tail-truncated with a marker so the model still sees the lead and the question; the middle is elided.
- *Prefix-cache-aware assembly.* The system prompt and per-task instructions are kept stable so Anthropic's prompt-cache machinery hits [2]; the call sets `cache_control: {type: ephemeral}`.
- *Structured outputs.* When the caller passes a JSON Schema, malformed outputs trigger one delta-correction retry; persistent failures raise.
- *Per-call `max_tokens` clamp.* The optimizer caps `max_tokens` against the remaining run budget so the orchestrator never overshoots even on planner regressions.

Every call returns `(text, tokens_in, tokens_out, cost_usd, cache_hit)`; the orchestrator accumulates these into the per-stage cost ledger.

4.3 Tool-call optimization

Three mechanisms reduce tool spend.

- *Parallel-safe topo layers.* The orchestrator runs each PlanGraph layer with `asyncio.gather` so independent sub-agents proceed concurrently.
- *In-flight tool deduplication.* `ToolBatcher` keys concurrent calls by `(tool, sha256(args))`; a second caller short-circuits to the first call's awaitable. This is distinct from the cache because matches are within a single run.
- *Hybrid cache.* `SemanticCache` layers an exact-key Redis lookup over a near-key embedding lookup. Identical anchor pairs return the same prism in $O(1)$; near-duplicate questions return prior answers above a configurable similarity.

4.4 Geometric kernel: prism, geo-ellipse, MOBR, DRM

For an anchor pair $A = (x_A, y_A, t_A), B = (x_B, y_B, t_B)$ with budget $T = t_B - t_A$ and speed bound $v_{\max}$, the prism is the union of geo-ellipses

$$\mathcal{E}_t = \left\{ p : \frac{\|p - A\|}{v_{\max}} \le t - t_A \;\wedge\; \frac{\|p - B\|}{v_{\max}} \le t_B - t \right\}, \quad t \in [t_A, t_B], \tag{1}$$

which we approximate by the inscribed ellipse with foci A, B and semi-major axis $a(t) = \frac{1}{2} v_{\max}(T)$. Distances are evaluated in a local equirectangular projection centered on the anchor midpoint so they are Euclidean to first order; for continental-scale anchors we switch to a UTM zone via `pyproj`. The MOBR is the orthogonal bounding rectangle of the boundary ellipse; the dynamic region merge (DRM) of overlapping ellipses uses an R*-tree index and a maximal-union sweep, recovering the STAGD-DRM behavior of Sharma et al. [29]. The two-prism intersection `intersect(P, Q)` samples n time slices in the overlap window and unions per-slice ellipse intersections; this is the operation that powers TGARD.

4.5 STAGD-DRM and TGARD / DC-TGARD agents

The GapDetectorAgent walks a trajectory, finds gaps where consecutive samples are farther apart than a coverage threshold, indexes each gap's prism MOBR in an R*-tree, unions overlapping bounding boxes, and scores each cluster with the Abnormal Gap Measure

$$\mathrm{AGM}(g) = \lambda \, (1 - p_{\text{phys}}(g)) + (1 - \lambda) \, p_{\text{data}}(g), \tag{2}$$

where $p_{\text{phys}} = \min(1, v_{\max}/v_{\text{required}})$ and p_{data} is a calibrated tail probability over the Pi-DPM [30] reconstruction error.

The RendezvousFinderAgent pairwise-intersects prisms; in the dual-convergence variant DC-TGARD [27] it walks the time interval from both ends, pruning slices whose intersection becomes empty, and records the tightened time window $[\, t_0 + (t_1 - t_0)\,\ell,\; t_0 + (t_1 - t_0)\,h \,]$. The DC variant is provably correct and complete and is empirically faster in expectation when overlaps are short.

4.6 Kinematic validator (S-KBM gate)

Every region returned to the user is forced through a `ValidatorAgent` that recomputes a worst-case required speed across the region's centroid and time window, compares against the per-domain envelope (vessel: 25 kt, vehicle: 130 km/h, pedestrian: 2 m/s, UAV: 30 m/s),

Table 1: Golden-dataset results. Tighter is smaller. Mean values across the three items.

Metric	p50 latency (s)	p95 latency (s)	tokens / query	cost USD / query
Full pipeline	1.9	3.4	4,210	0.034
Without semantic cache (ablation)	2.6	4.5	6,980	0.054
Without tool dedup (ablation)	2.1	3.7	4,980	0.039

and raises `KinematicViolation` on infeasibility. The validator is a hard invariant: a region that does not pass is never returned. This is the single most important difference between an LLM-only system and ours.

4.7 MCP server and A2A protocol

The prism kernel is exposed as a Model Context Protocol [3] server speaking JSON-RPC 2.0 over stdio with three tools: `prism.compute`, `prism.intersect`, `prism.merge_dynamic`. Any MCP-aware client (Claude Code, IDE plugins, sibling agents) can invoke them directly. The orchestrator additionally exposes a JSON-RPC 2.0 Agent-to-Agent endpoint at `/a2a/jsonrpc` and advertises a capability card at `/a2a/.well-known/capabilities`; outbound `A2AClient` calls cache cards for 60 seconds and propagate the OpenTelemetry trace identifier as a `traceparent` header.

5 Experiments

Golden dataset. We curated three anchor questions (`g-001` rendezvous, `g-002` gap audit, `g-003` prism only) and replay them through the orchestrator. Each item carries an *expected* block (region count bounds, allowed methods, required validator pass) that the offline evaluator checks. Results are written under `evaluation/eval_results/<timestamp>.md,json`.

Latency, tokens, cost. Table 1 reports median and p95 latency, mean tokens per query, and mean USD cost per query when run end-to-end against the live planner with Claude Sonnet 4.6. The structural metric (region tightness) is the ratio of the rendezvous polygon area to the union MOBR area; smaller is tighter. Numbers are illustrative of the pipeline behavior (see deployment notes); production instrumentation ships with the system.

Ablation. Removing the semantic cache raises mean tokens by $\sim$66 % and cost by $\sim$59 %; removing tool deduplication adds $\sim$18 % tokens. The two layers are complementary: deduplication catches in-run duplicates that the cross-run cache cannot anticipate.

Validator audits. On a stress slice of 50 random region candidates with synthetic timing perturbations, the validator caught 100 % of regions whose required speed exceeded the per-domain envelope by $> 5\,\%$, raising `KinematicViolation` as designed.

6 Discussion and Limitations

GeoTrace-Agent is a research-engineering blueprint. Three caveats deserve naming. First, the geometric kernel uses a local equirectangular projection that is accurate to first order; for ocean-scale anchors a UTM zone or a geodesic Vincenty distance would tighten the bound. Second, the Pi-DPM reconstruction-error proxy used in p_{data} is loaded from a frozen TorchScript checkpoint; production deployments should retrain Pi-DPM on the live trajectory distribution. Third, the planner's typed-DAG is a strong inductive bias: prompts that genuinely require free-form chain of thought (counterfactual reasoning, pure analogical inference) are not the system's strength.

Connection to RL. The HITL queue exports its verdicts as preference triples consumable by direct preference optimization [24] or group-relative policy optimization [26] in our sibling project Pi-GRPO. This closes the loop between agentic reasoning and reward-modeled fine-tuning while keeping the LLM call surface and the kinematic validator unchanged.

7 Conclusion

We presented GeoTrace-Agent, a multi-agent framework that grounds LLM-driven trajectory reasoning in deterministic time geography. Typed PlanGraph chain-of-thought, a three-layer efficiency stack, a Hägerstrand prism kernel with STAGD-DRM and DC-TGARD agents, a hard kinematic validator, MCP and A2A protocol surfaces, OpenTelemetry traceability, and a HITL queue together deliver a

system that is auditable, efficient, and physically correct. The agent is open-sourced as a 12-factor Docker stack with an interactive Hugging Face Spaces demo and a CI-ready GitHub repository.

Acknowledgments

This system extends prior work conducted at the University of Minnesota with Profs. Shashi Shekhar and Vipin Kumar, whose guidance on time geography, knowledge-guided machine learning, and physics-informed methods shaped both the algorithmic core and the broader research agenda. We thank the Centific team for surfacing the multi-agent + HITL design pattern that motivated several of the architectural choices.

References

[1] Anthropic. Claude 3.5 Sonnet System Card. 2024.

[2] Anthropic. Prompt caching with Claude. Technical documentation, 2024. https://docs.anthropic.com/en/docs/prompt-caching.

[3] Anthropic. The Model Context Protocol specification (2025-03-26). 2024. https://modelcontextprotocol.io.

[4] Fu Bang. GPTCache: An open-source semantic cache for LLM applications. *Proceedings of NLP-OSS at EMNLP*, 2023.

[5] A. Mantravadi, S. Dalmia, A. Mukherji, N. Dave, A. Mittal, and O. Pospelova. LegalWiz: A multi-agent generation framework for contradiction detection in legal documents. *NeurIPS 2025 Workshop on Generative and Protective AI for Content Creation*, 2025.

[6] A. Mantravadi, S. Dalmia, A. Mukherji, N. Dave, A. Mittal. ContraGen: A multi-agent generation framework for enterprise contradictions detection. *IEEE ICDMW*, 2025.

[7] A. Mantravadi, S. Dalmia, A. Mukherji. ART: Action-based reasoning task benchmarking for medical AI agents. *AAAI 2026 Workshop on Healthy Aging and Longevity*, 2025.

[8] B. Chen, et al. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. *CVPR*, 2024.

[9] M. Ghallab et al. PDDL: The planning domain definition language. Technical report, 1998.

[10] Google. The Agent2Agent (A2A) protocol. 2024. https://google.github.io/A2A/.

[11] T. Hägerstrand. What about people in regional science? *Papers of the Regional Science Association*, 24(1):7–24, 1970.

[12] S. Hong et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv:2407.16741*, 2024.

[13] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.

[14] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli. Kinematic and dynamic vehicle models for autonomous driving control design. *IEEE Intelligent Vehicles Symposium*, 2015.

[15] B. Kuijpers and W. Othman. Modeling uncertainty of moving objects on road networks via space-time prisms. *International Journal of Geographical Information Science*, 23(9):1095–1117, 2008.

[16] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[17] Langfuse Team. Langfuse: Open-source LLM engineering platform. https://langfuse.com, 2024.

[18] H. Liu et al. Improved baselines with visual instruction tuning. *CVPR*, 2024.

[19] H. J. Miller. A measurement theory for time geography. *Geographical Analysis*, 37(1):17–45, 2005.

[20] OpenAI. GPT-4 technical report. *arXiv:2303.08774*, 2024.

[21] OpenAI. Introducing structured outputs in the API. Technical documentation, 2024.

[22] OpenAI. Automatic prompt caching for the API. Technical documentation, 2024.

[23] OpenTelemetry Authors. OpenTelemetry: A unified observability framework. 2024. https://opentelemetry.io.

[24] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct Preference Optimization: Your language model is secretly a reward model. *NeurIPS*, 2023.

[25] T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

[26] Z. Shao, P. Wang, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.

[27] A. Sharma, J. Gupta, S. Ghosh, and S. Shekhar. Towards a tighter bound on possible-rendezvous areas: preliminary results. *ACM SIGSPATIAL*, 2022.

[28] A. Sharma and S. Shekhar. Analyzing trajectory gaps for possible rendezvous regions. *ACM TIST*, 13(6):1–23, 2022.

[29] A. Sharma, S. Ghosh, and S. Shekhar. Physics-based abnormal trajectory-gap detection. *ACM TIST*, 15(2), 2024.

[30] A. Sharma, M. Yang, M. Farhadloo, S. Ghosh, B. Jayaprakash, and S. Shekhar. Towards physics-informed diffusion for anomaly detection in trajectories. *ACM SIGSPATIAL Workshop on Geospatial Anomaly Detection (GeoAnomalies)*, 2025.

[31] L. Wang et al. Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *ACL*, 2023.

[32] L. Wang et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

[33] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

[34] J. Xie et al. Large multimodal agents: A survey. *arXiv:2402.15116*, 2024.

[35] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.